Date: __/__/__ Day _____

CHAPTER : 6

★ "STACKS (LIFO)" ★

STACKS :-

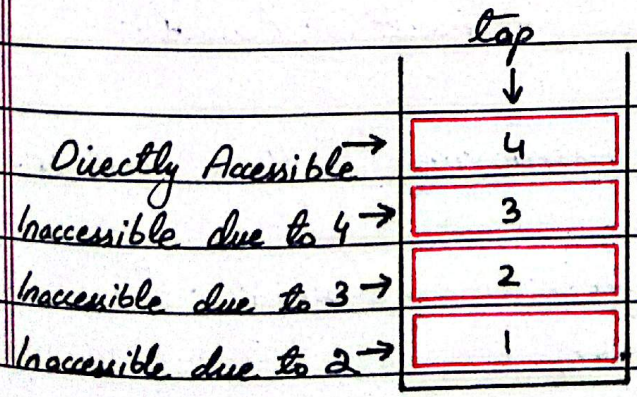
Definition :-

"Stack is a bucket like data structure in which successive data is put on top of existing data. The underlying data is inaccessible until data on top of it is removed."

Explanation :-

Stack basically follows the "Last in first out" protocol, also called "LIFO". Meaning the last data entry will be removed first.

MEMORY DIAGRAM :-



Date: __/__/__ Day

A stack can implemented using the previously learnt vectors or linked lists, it can also be implemented using another data structure called "deque", will look into this later on, for now we will use vectors and lists.

STACK USING VECTOR:-

A stack mainly has only three methods:-

1) push(); "Used to insert value on top."
2) top(); "Used to access value on top."
3) pop(); "Used to remove value from top."

Now if we cutoff the access of indexing in a vector and also cut off the access to methods like:-

1) front();
2) data();
3) clear();
4) insert(); /erase();

And also removing other methods:-

1) begin(); 3) rbegin(); 5) assign();
2) end(); 4) rend();

Date:__/__/__ Day_____

And the last step:-
Name push_back() as push(), back() as top() and pop_back() as pop().

Now what you have is basically a stack implemented using a vector. A stack is basically a restricted vector which allows the use of "LIFO". This also means that everything you are doing with a stack can be done using a vector. Let's take a look at stack's code:-

Code:-

```
class stack{
private:
std::vector<int> v;
public:
    void push(int n){ v.push_back(n); }
    void pop(){ v.pop_back(); }
    int top(){ return v.back(); }
    bool empty(){ return v.empty(); }
};
```

In order for this to work, you need to include vectors using:-
#include<vector>.

Date: __/__/__ Day

STACK USING LIST :-

Now using the cut off logic, we can implement stack using a list.

Consider a list l, if we cut off the access to its methods like push_back() and pop_back(). We will basically be left with push_front() and pop_front(). We know that head of a linked list updates with every push_front() to the inserted node and similary it goes back to last head (previous head) with every pop_front(). So we can derive the top() logic from head as well by naming push_front() as push(), pop_front() as pop() and using front() as top().

Other than this, we will cut off the access to all methods of a linked list which can be used to violate the "LIFO" protocol of a stack. Let's take a look at the code :-

Date: / / Day

Code :-

```
class stack{
private:
    std::list<int> l;
public:
    void push(int n){ l.push_front(n); }
    void pop(){ l.pop_front(); }
    int top(){ return l.front(); }
    bool empty(){ return l.empty(); }
};
```

In order for this to work, you need to include lists using:

```
#include <list>
```

C++ STL :-

All the data structures that we studied till now such vectors, lists, stacks and many which we will study ahead are already defined in C++ STL and can be used anytime without building from scratch using :-

```
#include<vector>
#include<list>
#include<stack>
```

But DSA is the journey of

Date: __/__/__ Day

understanding the internal working of data structures, this is why, for the duration of this course we are making these from scratch.

See how we first learnt the implementation of vectors and lists before, but now that we know how they work, we just used their libraries to implement stack in both manners.

Similarly, the up-coming data structures that we will learn are also going to be a part of C++ STL, but before using them, we must learn and understand them first.

For now this is gonna be all for stacks, however I will give some practice problems for getting a better grip on stacks manipulation and handling. Keep in mind that you can use all your knowledge from previous chapters and you can also use C++ STL to

Date:__/__/__ Day_____

solve these questions

PRACTICE EXERCISE :-

1) Make a function called "reverseStack" with returntype "void" which takes a "stack<int> s" in argument that is passed by reference. Your function should reverse all the values in the stack. (Use of C++ STL is allowed).
2) Determine the time complexity of push(), pop() and top() functions.
3) Make a "print" function which takes a "stack<int> s" in argument, and prints all the values of the stack, its return type should be void. (Use of C++ STL is allowed).

This is it for this chapter. This was all about stacks for now. As always, the codes can be found in github repo:- "github.com/coder-Retro/Fast-University"

Go into DSA > Stack, Next up we will dive into the "Queue", which is another data structure similar to stack but a bit different. Till Then GOOD LUCK!